

DocuMind AI

Adaptive Multimodal RAG for Source-Grounded PDF Intelligence

Supervisor	Eng / Jana Hatem
Assistant Engineer	Gad Amr
Team	Kemet AI
Deliverables	Documentation, Presentation, Web Application

Built around trust, evidence, hallucination control, and education.

Project Metadata

- Project name: DocuMind AI
- Project type: Adaptive multimodal RAG platform for PDF intelligence
- Repository: adaptive-rag-platform
- Supervisor: Eng / Jana Hatem
- Assistant Engineer: Gad Amr
- Team: Kemet AI
- Primary deployment target: Railway
- Backend: FastAPI, Python, SQLAlchemy, PostgreSQL
- Frontend: React, Vite, TypeScript, Tailwind CSS
- LLM provider: Groq through environment variable configuration

Executive Summary

DocuMind AI is a production-oriented Retrieval-Augmented Generation platform that allows users to upload PDF sources and ask questions that must be answered only from the uploaded documents. The system indexes the uploaded source, retrieves relevant evidence, generates grounded answers with page-level citations, measures hallucination risk, and refuses unsupported questions instead of guessing.

The platform also includes an Exam Studio that can generate structured exams from the uploaded source, making it useful for educators, university doctors, teaching assistants, trainers, and students.

Problem Statement

Standard PDF question-answering systems often fail when documents contain more than plain text. Academic and technical PDFs usually include tables, scanned pages, charts, diagrams, figures, and multi-page context. A basic text-only RAG pipeline can lose important evidence, hallucinate unsupported answers, or fail to explain where an answer came from.

DocuMind AI addresses this by combining document analysis, hybrid retrieval, source-only answer generation, citation verification, conversation memory, and structured exam generation in one application.

Core Objectives

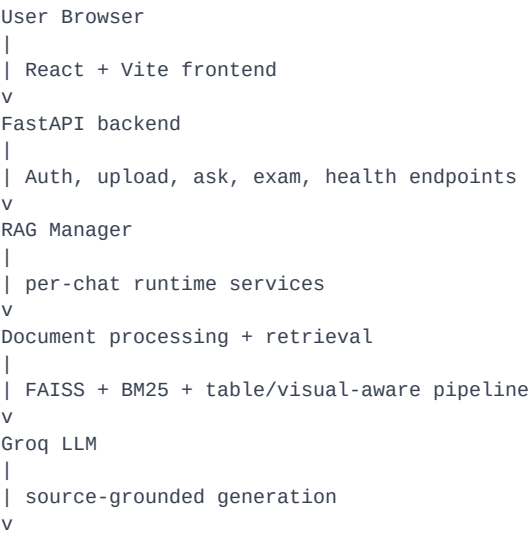
- 1 Allow users to upload PDFs and query them through a clean web interface.
- 2 Ensure every answer is grounded in the uploaded source only.

- 3 Refuse unsupported questions with a helpful specialist-consultation message.
- 4 Provide evidence pages, confidence, groundedness score, and hallucination risk.
- 5 Support structured exam generation from the uploaded PDF.
- 6 Persist uploaded PDFs so indexes can be rebuilt after server restarts.
- 7 Provide authentication and account isolation through database-backed users.
- 8 Deploy as a production-ready Railway service.

User Roles

Role	Use Case
Student	Upload lecture notes or books and ask source-grounded questions.
Doctor / University professor	Generate exams from uploaded course material.
Teaching assistant	Build quizzes, review source evidence, and prepare study material.
Researcher	Summarize and inspect PDF sources without relying on outside knowledge.
Admin / developer	Deploy, monitor, configure, and extend the system.

High-Level Architecture



```
Structured API response
|
| answer, evidence, sources, confidence, hallucination risk
v
Frontend UI
```

Repository Structure

```
adaptive-rag-platform/
Dockerfile
railway.json
README.md
requirements.txt
docs/
PROJECT_DOCUMENTATION.md
presentation/
backend/
requirements.txt
documind_rag/
app/
main.py
schemas.py
controllers/
core/
models/
repositories/
services/
rag/
service.py
pipeline.py
notebook_core.py
frontend/
package.json
src/
pages/
services/
components/
types/
public/
team/
```

Backend Architecture

The backend follows a modular FastAPI structure:

- **Controllers:** HTTP endpoints for auth, upload, ask, exam, health, and index rebuild behavior.
- **Schemas:** Pydantic request/response models for stable API contracts.
- **Core:** Configuration, database engine, security, and dependency providers.
- **Models:** SQLAlchemy database models such as users and persisted document sources.
- **Repositories:** Database access logic for users and uploaded documents.
- **Services:** Authentication and RAG orchestration logic.
- **RAG layer:** Notebook-derived pipeline code wrapped behind a production service facade.

Backend API Endpoints

Endpoint	Method	Purpose	Auth
/auth/signup	POST	Create a user account	No
/auth/login	POST	Login and receive JWT token	No
/health	GET	Return backend and runtime index health	No
/chats/{chat_id}/sources	POST	Upload PDF files and build indexes	Yes
/ask	POST	Ask a question against a chat source	Yes
/exam	POST	Generate a structured exam from source	Yes
/chats/{chat_id}/memory	GET	Return bounded conversation memory	Yes

Important Pydantic Schemas

AskRequest

```
{
  "query": "What does the source say about treatment?",
  "chat_id": "source-id",
  "document_ids": null,
  "verbose": false,
  "use_memory": true
}
```

AskResponse

```
{
  "answer": "Grounded answer with citations.",
  "query": {
    "original": "string",
    "rewritten": "string",
    "language": "ar | en",
    "query_type": "string",
    "filters": {},
    "search_queries": []
  },
  "confidence": 0.91,
  "hallucination_risk": 0.0,
  "groundedness_score": 1.0,
  "sources": [12, 18],
  "evidence": []
}
```

```
}
```

ExamResponse

```
{
  "title": "Source-grounded exam",
  "questions": [
    {
      "type": "mcq",
      "question": "Question text",
      "options": ["A", "B", "C", "D"],
      "answer": "Correct answer",
      "explanation": "Why this answer is correct",
      "page": 12,
      "section": "Source section"
    }
  ],
  "confidence": 0.9,
  "hallucination_risk": 0.0,
  "groundedness_score": 1.0,
  "sources": [12]
}
```

RAG Pipeline

The RAG pipeline is designed to support heterogeneous PDF content:

- 1 PDF ingestion: User uploads one or more PDF files.
- 2 Document analysis: Pages and chunks are extracted from the PDF.
- 3 Index building: Runtime indexes are built for semantic and keyword retrieval.
- 4 Query rewriting: The user query is normalized and rewritten for better retrieval.
- 5 Query classification: The system detects whether the query is text, table, chart, image, summarization, or cross-page.
- 6 Hybrid retrieval: FAISS and BM25 retrieve candidate evidence.
- 7 Reranking: Candidate chunks are re-scored before generation.
- 8 Context building: Retrieved evidence is assembled into source-only context.
- 9 LLM generation: Groq LLM generates the answer using only retrieved evidence.
- 10 Verification: Claims and citations are checked to compute hallucination risk and groundedness.

Source-Only Answer Policy

The platform is intentionally restricted to the uploaded PDF source. It should not answer from general world knowledge or internet knowledge.

If the uploaded PDF does not contain enough evidence, the backend returns a refusal message instead of fabricating an answer. The refusal tells the user to consult a doctor for medical questions or a qualified specialist for other domains.

This behavior is implemented in:

- backend/documind_rag/rag/service.py
- backend/documind_rag/rag/notebook_core.py
- frontend/src/pages/AssistantPage.tsx

Hallucination Control

The system tracks answer reliability using:

- Confidence score: Estimated answer quality.
- Groundedness score: How well the answer is supported by retrieved evidence.
- Hallucination risk: Fraction of unsupported checked claims.
- Evidence list: Retrieved pages/chunks supporting the answer.
- Source pages: Page numbers used by the final answer.

Unsupported or weakly supported responses are blocked or marked as insufficient evidence.

Persistent Indexing Strategy

FAISS is used as a fast runtime vector index. Runtime indexes can be lost when the Railway container restarts, so the system persists the original uploaded PDF bytes in PostgreSQL.

When a user asks a question or generates an exam after a restart:

- 1 The backend detects that the runtime index is missing.
- 2 It loads the persisted PDF from PostgreSQL.
- 3 It rebuilds the FAISS/BM25 indexes automatically.
- 4 It retries the original request.

This provides production resilience without replacing FAISS with a heavier vector database during the hackathon phase.

Relevant files:

- backend/documind_rag/app/models/document_source.py
- backend/documind_rag/app/repositories/document_repository.py
- backend/documind_rag/app/controllers/index_rebuild.py
- backend/documind_rag/app/controllers/assistant_controller.py
- backend/documind_rag/app/controllers/exam_controller.py

Authentication and Database

Authentication uses:

- User signup and login.
- Password hashing.
- JWT bearer tokens.
- SQLAlchemy models.
- PostgreSQL in production.
- SQLite fallback for local development.

The database stores:

- User accounts.
- Persisted uploaded PDF sources.

Frontend Features

The frontend provides:

- Landing page with dark/light theme toggle.
- Language toggle on the landing page.
- Mobile-responsive horizontal card rails.
- Authentication pages.
- Document upload workspace.
- Source-grounded chat assistant.
- Evidence panel with page-level traces.
- Exam Studio for source-based exam generation.
- Analytics dashboard.
- Voice input using browser Web Speech API.
- Team section with member photos.

Exam Studio

Exam Studio allows educators to generate assessments from uploaded source material. Users can configure:

- Source document.

- Number of questions.
- Difficulty.
- Question types.
- Optional focus topic.

The backend returns structured Pydantic exam output with answer keys, explanations, page references, groundedness score, and hallucination risk.

Deployment

The root Dockerfile builds both frontend and backend:

- 1 Node builds the Vite frontend.
- 2 Python image installs backend dependencies.
- 3 Tesseract and Arabic OCR packages are installed.
- 4 Frontend static files are copied into the backend image.
- 5 FastAPI serves both API and frontend SPA from one Railway service.

Railway uses:

- Dockerfile
- railway.json
- /health as healthcheck path
- PORT environment variable

Required Environment Variables

Variable	Purpose
GROQ_API_KEY	Groq LLM API key
DATABASE_URL	PostgreSQL connection URL in production
JWT_SECRET_KEY	Secret used to sign JWTs
ACCESS_TOKEN_EXPIRE_MINUTES	Token lifetime
DOCUMIND_CORS_ORIGINS or FRONTEND_ORIGINS	Allowed frontend origins
DOCUMIND_LOW_MEMORY	Enables low-memory RAG mode

DOCUMIND_MAX_ANSWER_TOKENS	Controls answer/exam generation length
DOCUMIND_AUTO_BUILD	Optional startup auto-build from configured PDFs
DOCUMIND_PDF_PATHS	Optional startup PDF paths

Local Development

Backend

```
cd backend
pip install -r requirements.txt
uvicorn documind_rag.app.main:app --reload
```

Frontend

```
cd frontend
npm install
npm run dev
```

Production Build

```
cd frontend
npm run build
```

Testing and Verification

Current verification flow:

- Python syntax verification with `python -m compileall`.
- TypeScript verification with `tsc --noEmit`.
- Frontend production build with `vite build`.
- Production smoke checks:
- `/health`
- frontend root `/`

Recommended next testing additions:

- Unit tests for repositories and services.
- API integration tests for auth, upload, ask, and exam.
- RAG evaluation set using known PDF questions and expected answers.
- Frontend Playwright smoke tests for upload, ask, and exam generation.

Current Strengths

- Clean separation between frontend and backend.
- Production deployment through a single Railway service.
- Database-backed user authentication.
- Persistent uploaded sources with automatic index rebuild after restart.
- Source-only answer policy.
- Hallucination risk and groundedness scoring.
- Structured Pydantic schemas.
- Responsive UI and mobile landing page.
- Voice input support.
- Educator-focused Exam Studio.

Known Limitations

- FAISS indexes are rebuilt in memory after restart rather than stored as persisted vector indexes.
- Rebuild time depends on PDF size and complexity.
- Web Speech API support varies by browser.
- Full Arabic/English localization is not yet complete; language toggle currently controls page direction and user preference.
- OCR quality depends on PDF scan quality.

Recommended Roadmap

- 1 Add full i18n translation for the frontend.
- 2 Add background indexing jobs for large PDFs.
- 3 Add progress streaming during index rebuild.
- 4 Add a persistent vector database option such as pgvector or Qdrant for larger production scale.
- 5 Add admin dashboard for usage and system health.
- 6 Add automated evaluation datasets for hallucination and groundedness.
- 7 Add export for generated exams as PDF/DOCX.
- 8 Add role-based permissions for teachers, students, and admins.

Conclusion

DocuMind AI is a strong hackathon-ready platform because it solves a real problem: making PDF-based AI answers verifiable, source-only, and useful for education. The architecture is already production-aware, with authentication, deployment configuration, persistent source storage, structured APIs, and a modern responsive frontend.